

Projet Mesurer son Environnement II

2025-2026 | Dr. Ing. Maxime SPIRLET



Projet Mesurer son Environnement

Partie 2

■ Planning

Etude du fonctionnement des capteurs enviro+

Traitement et fusion de données

Utilisation et fonctionnement du protocole MQTT

Utilisation de l'interface NodeRed

Réalisation du projet final par groupe de 2

Rédaction du rapport

Campagne de mesures dans la classe (1h)

Analyse et interprétation des résultats



CAPTEURS

Capteurs (MSE I)

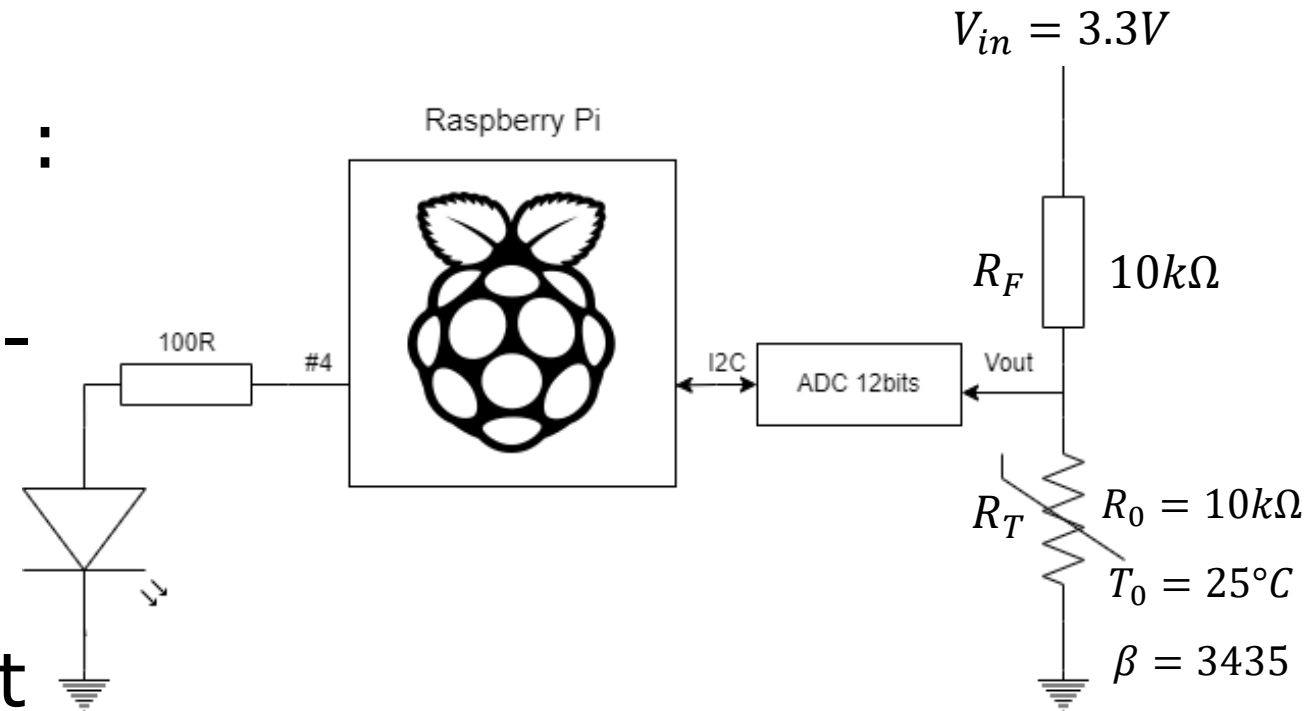
Vous connaissez déjà un capteur passif analogique : la thermistance.

- La relation de Steinhart-Hart:

$$R_T = R_0 e^{\beta \left(\frac{1}{T} - \frac{1}{T_0} \right)}$$

- Un pont diviseur permet de déduire la valeur de R_T en mesurant la tension V_{out} avec un ADC :

$$V_{out} = V_{in} \frac{R_T}{R_T + R_F}$$



Capteurs (MSE II)

Découverte de 6 nouveaux capteurs actifs :

- BME280 (Enviro+),
- Micro (Enviro+),
- PM5003 (Enviro+),
- LTR-559 (Enviro+),
- LMT84 (Breadboard),
- SCD30 (Breadboard).



CAPTEURS

BME280

BME280

- Il s'agit d'un capteur de pression absolue et d'humidité relative. Le BME280 est aussi équipé d'un capteur de température dont la fonction première est la compensation de la mesure de pression et d'humidité. Néanmoins, on peut l'utiliser pour obtenir une estimation de la température ambiante.
- Il s'agit d'un capteur basse consommation, haute précision et haute linéarité de mesure.

1. Pression :

- Plage de mesure : 300 – 1100hPa (ISA* = 1013hPa/15°C @ 0 MSL),
- Erreur de Sensibilité : $\pm 0,25\%$ $\rightarrow 1m$ d'altitude,
- Coef. de température : $\pm 1,5Pa/K$,

2. Humidité :

- Plage de mesure : 0 $\rightarrow$ 100%,
- Précision : $\pm 3\%$ d'humidité relative,

3. Température :

- Plage de mesure : $-40^{\circ} - 85^{\circ}C$,
- Précision : $\pm 1^{\circ}C$,

■ Consommation :

- 3,6 μA @ 1Hz (P, H, T) (3,3V $\rightarrow$ 12 μW).



BME280 - schéma bloc

3.1 Block diagram

Figure 2 shows a simplified block diagram of the BME280:

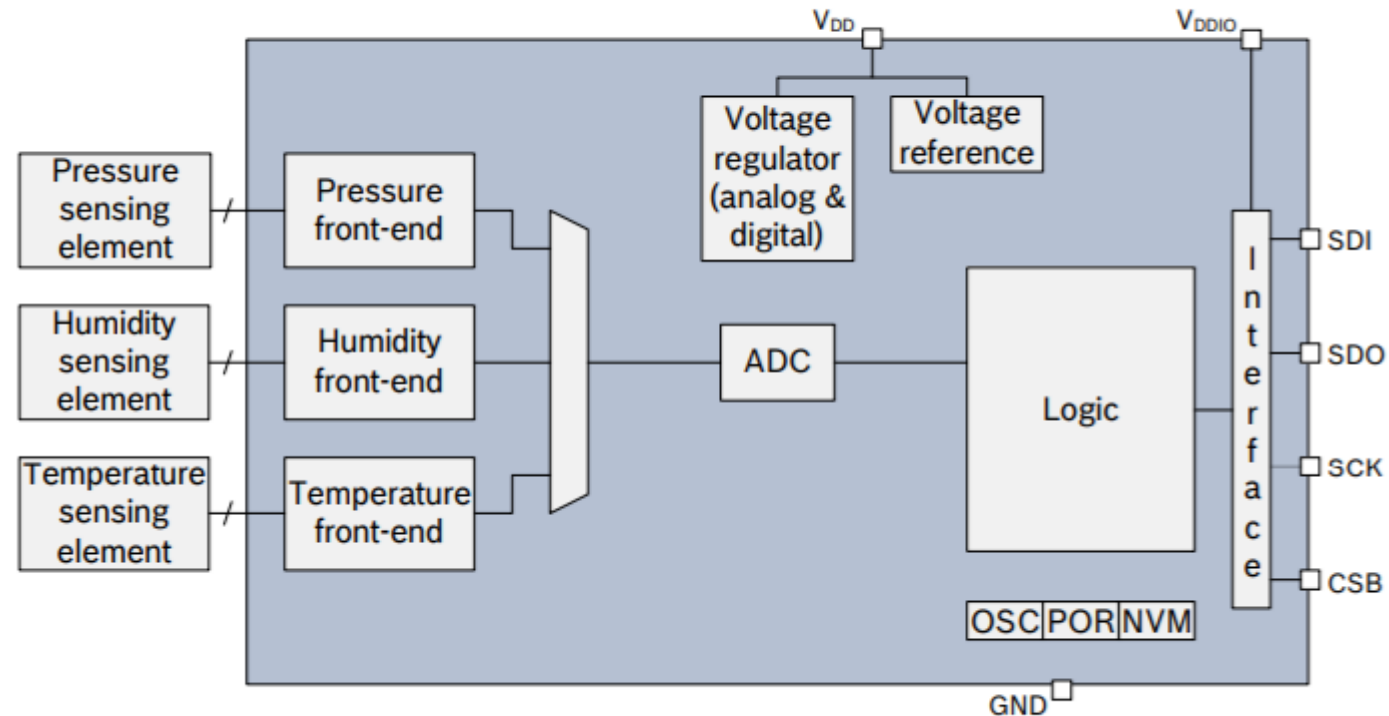
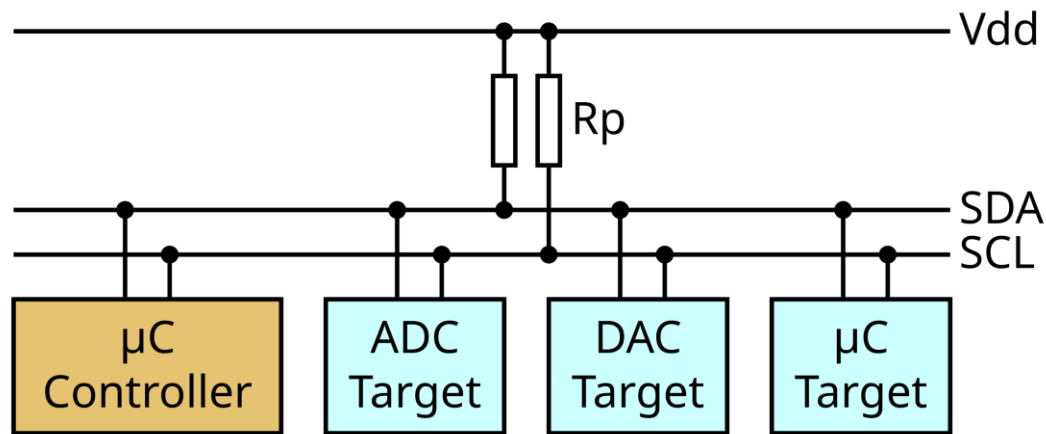


Figure 2: Block diagram of BME280

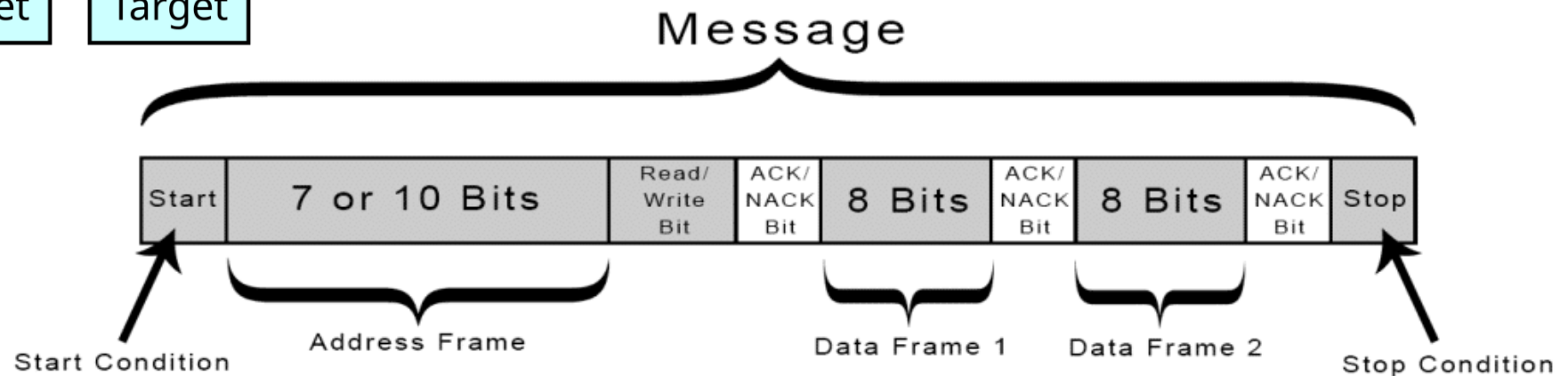
BME280 – I²C*

La communication entre le BME280 et le Raspberry Pi s'effectue au travers d'un bus I²C. L'I²C utilise :

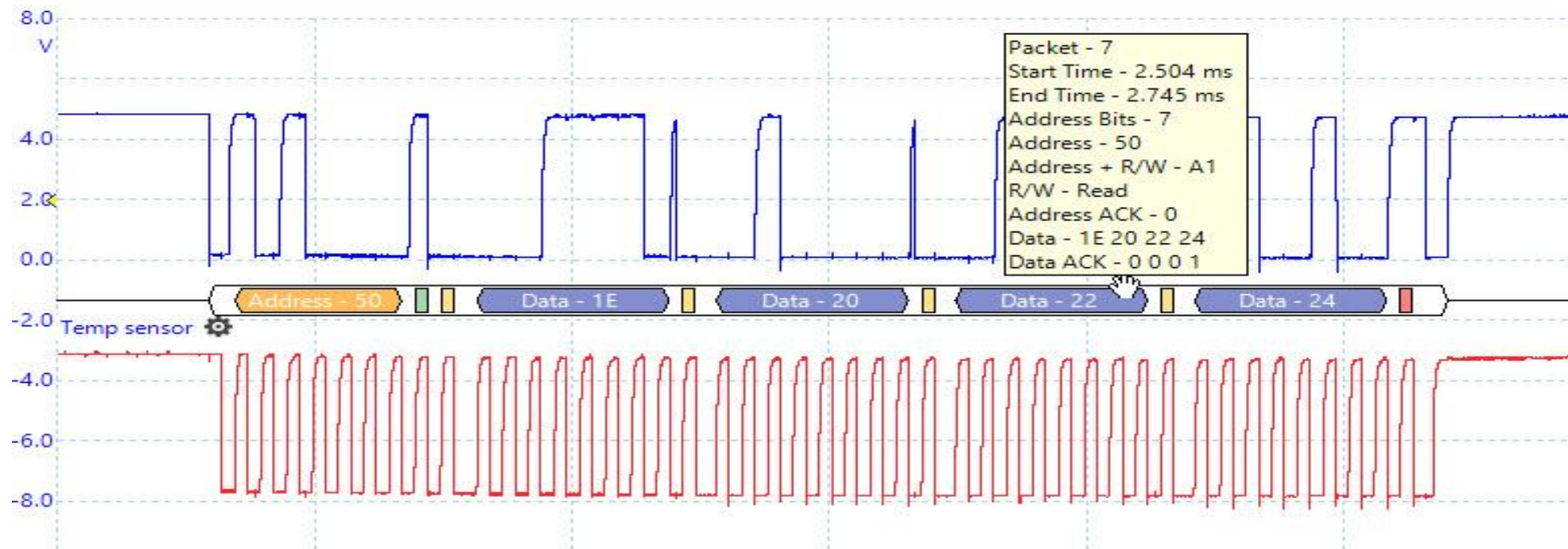
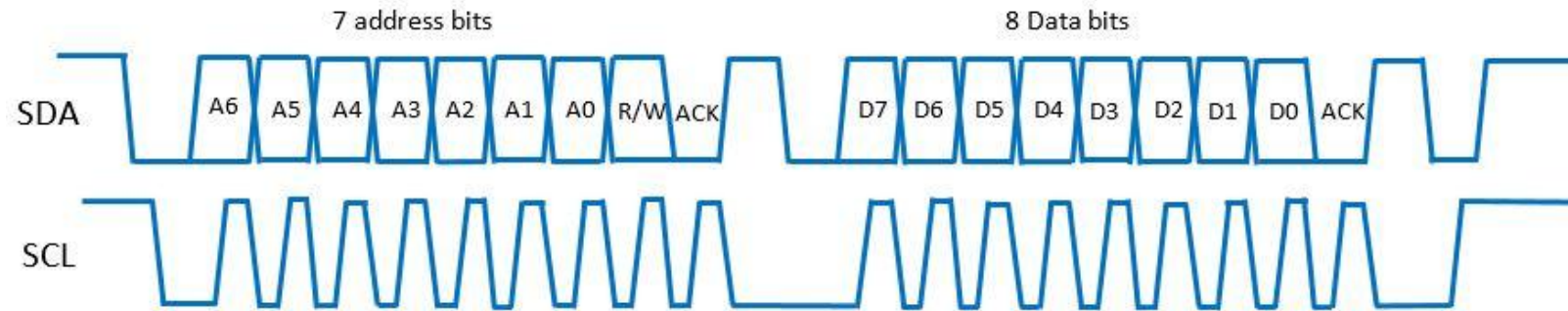
- deux lignes pour la communication, SDA (data) et SCL (clock),
- un système d'adresse afin de communiquer entre le contrôleur et une cible sur le bus.



Topologie



I2C – Waveform



BME280 - Python

```
from bme280 import BME280
import time

bme280 = BME280()                                #enable BME280 sensor

while True:
    temp = bme280.get_temperature()               #Get temperature in °C
    pres = bme280.get_pressure()                  #Get pressure in hPa
    hum = bme280.get_humidity()                   #Get relative humidity in %Hr

    #print measurements
    print(f"T : {temp:.2f} °C")
    print(f"P : {pres:.2f} hPa")
    print(f"H : {hum:.2f} %")
    time.sleep(1)
```

CAPTEURS

Microphone

Microphone – MEMS*

- Le SPH0645LM4H-B est un microphone miniature à faible consommation d'énergie, avec une sortie numérique I2S (Inter-IC Sound).
- Il comprend un capteur acoustique SiSonic™ de haute performance, un convertisseur analogique-numérique en série, et une interface pour conditionner le signal au format 24 bits.
- L'interface I2S facilite l'intégration dans le système et permet une connexion directe aux processeurs numériques, aux processeurs d'application et aux microcontrôleurs.
- Éliminant ainsi le besoin d'un codec audio externe, le SPH0645LM4H-B est parfaitement adapté aux applications portables où la taille et la consommation d'énergie sont des contraintes.

*Les **MEMS** (Micro-Electro-Mechanical Systems), ou systèmes microélectromécaniques en français, sont des dispositifs qui combinent des composants mécaniques et électroniques à une échelle microscopique.

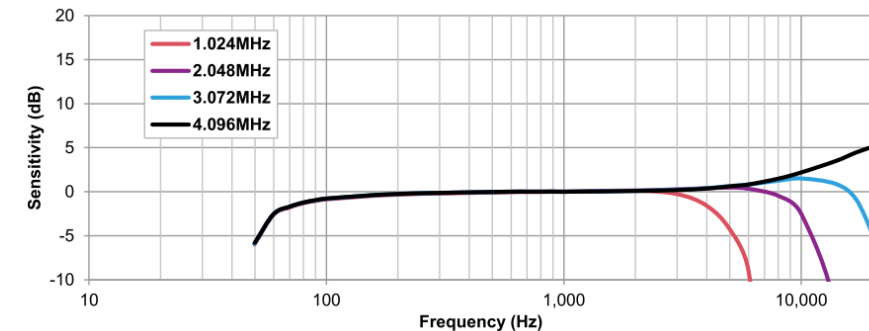
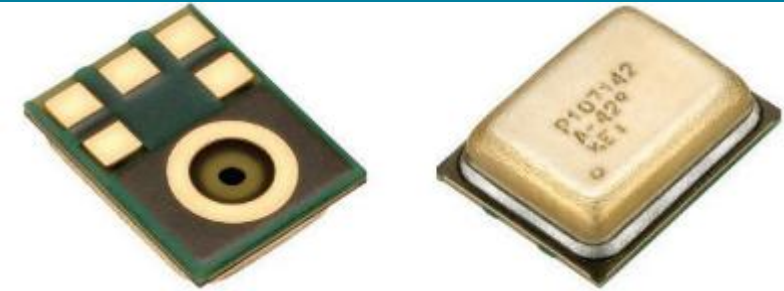


Figure 3: Typical Free Field Response Normalized to 1 kHz

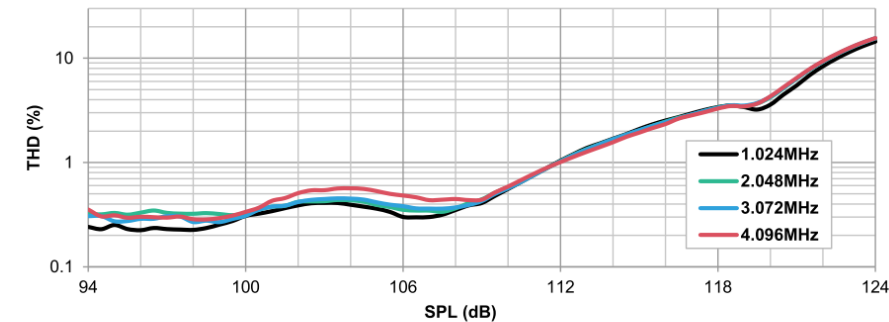


Figure 4: Typical THD vs Input SPL

Microphone – MEMS

■ Bruit par bandes de fréquence arbitraires :

```
from enviroplus.noise import Noise
noise = Noise()
while True:
    amps = noise.get_amplitudes_at_frequency_ranges([
        (100, 200),
        (500, 600),
        (1000, 1200)
    ])
    print(amps)
```

Renvoie l'amplitude moyenne des fréquences dans les plages de fréquence données.

■ Bruit par bandes de fréquence préselectionnées:

```
from enviroplus.noise import Noise
noise = Noise()
while True:
    low, mid, high, all = noise.get_noise_profile()
    print(low)           # 0 % Fmax -> 12% Fmax
    print(mid)           # 12% Fmax -> 36% Fmax
    print(high)          # 52% Fmax -> 100% Fmax
    print(all)           # 0 % Fmax -> 100% Fmax
```

Renvoie un profil caractéristique du bruit tel que toutes les fréquences sont regroupées en 3 groupes pondérés exprimés en pourcentage de la gamme de fréquences totale ($\approx 10\text{ Hz} \leftrightarrow 20\text{ kHz}$).

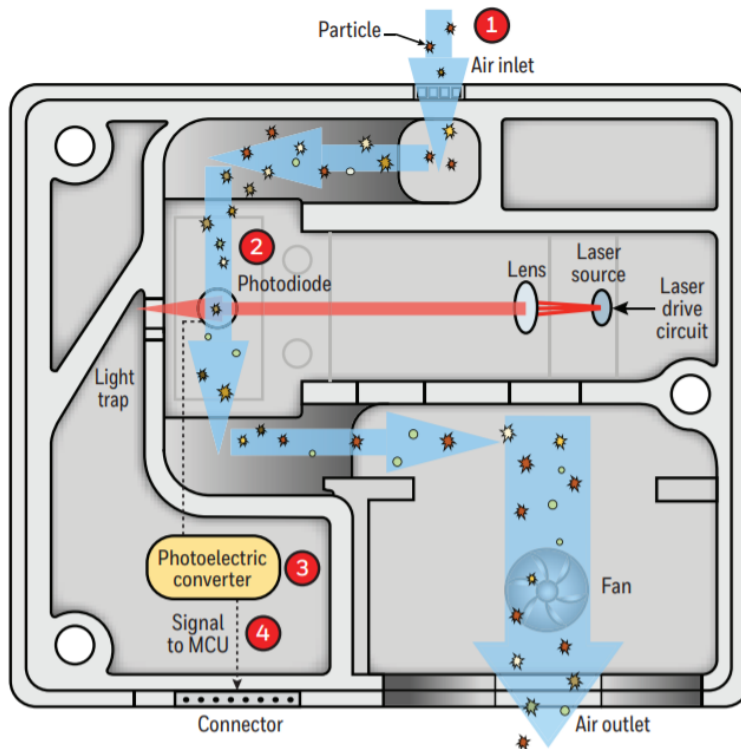
CAPTEURS

PMS5003

Le capteur PMS5003

Principe de fonctionnement :

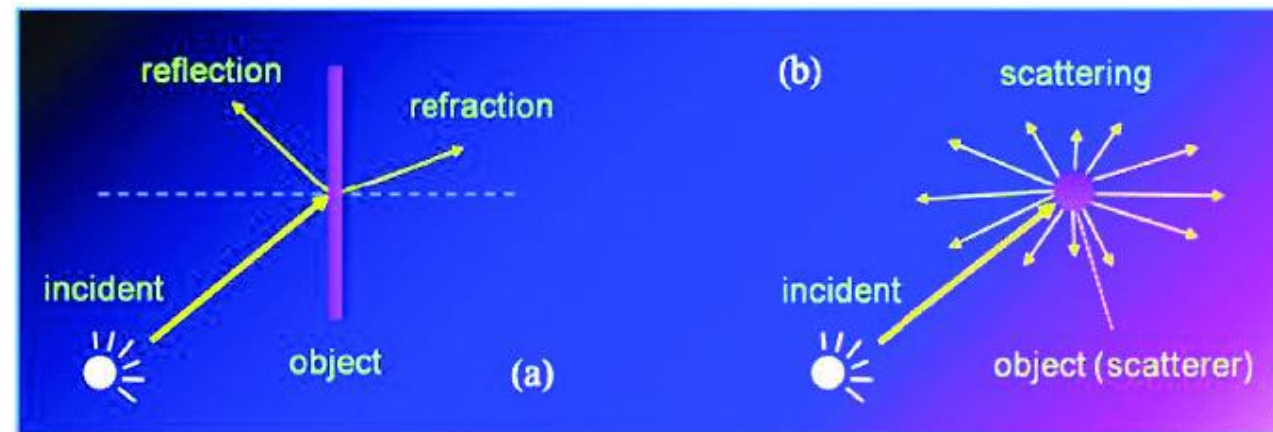
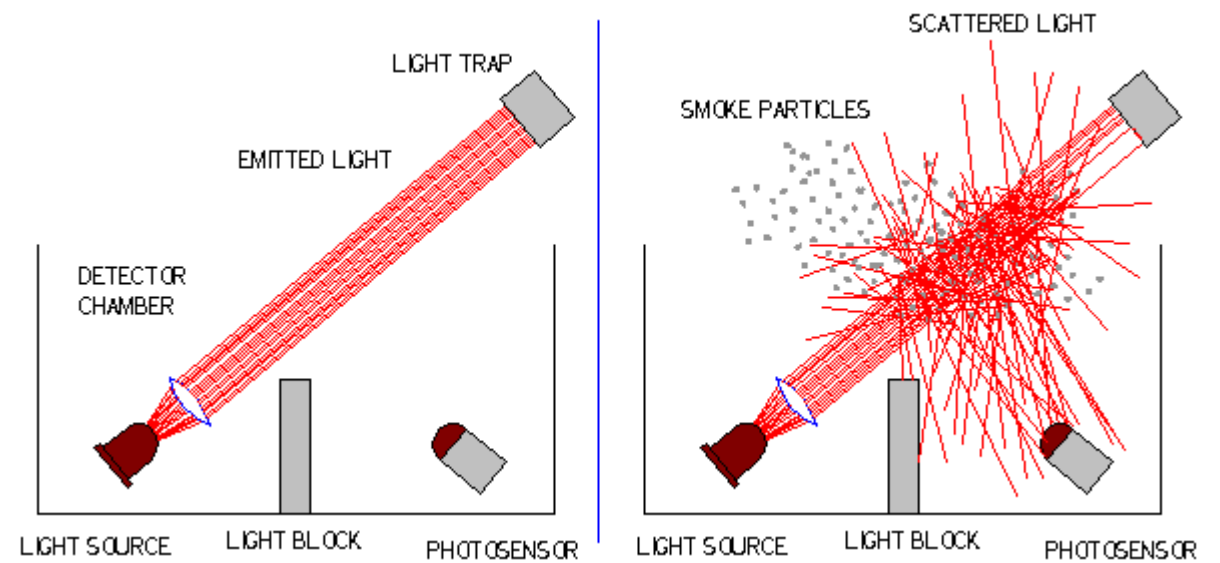
Mesurer la diffusion (scattering) d'un rayon laser qui est proportionnelle à la quantité de particules en suspension dans l'air. Plus le rayon est diffus plus la concentration en particules fines est élevée.



- 1 Fan draws in air through inlet.
- 2 Air passes through the laser where the light reflected off the particles is captured by the photodiode.
- 3 The photodiode passes information to the photoelectric converter. The photoelectric converter processes the signal from the particles into density.
- 4 Signal is transmitted to micro control unit where a proprietary algorithm processes the data and supplies outputs for the density of the particulate ($\mu\text{g}/\text{m}^3$).

Light Scattering

- Ce type de capteur établit une courbe de variation de diffusion de la lumière en fonction du temps. Il est alors possible de calculer la quantité et le diamètre des particules présentes.



PMS5003 - Datasheet

- Ce capteur mesure:
 - selon deux standards, la concentration en :
 - PM1 : quantité de particules de $1\mu\text{m}$ exprimé en $\mu\text{g}/\text{m}^3$,
 - PM2,5 : quantité de particules de $2,5\mu\text{m}$ exprimé en $\mu\text{g}/\text{m}^3$,
 - PM10 : quantité de particules de $10\mu\text{m}$ exprimé en $\mu\text{g}/\text{m}^3$,
 - La quantité de particule dans 0,1L d'air des diamètres au delà de :
 - $0,3\mu\text{m}$,
 - $0,5\mu\text{m}$,
 - $1\mu\text{m}$,
 - $2,5\mu\text{m}$,
 - $5\mu\text{m}$,
 - $10\mu\text{m}$,
- La plage effective de mesure des PM2,5 est de $500\mu\text{g}/\text{m}^3$,
- La résolution est de $1\mu\text{g}/\text{m}^3$,
- L'erreur de mesure (standard PM2,5) est de :
 - $\pm 10\% @ 100 \rightarrow 500\mu\text{g}/\text{m}^3$,
 - $\pm 10\mu\text{g} @ 0 \rightarrow 100\mu\text{g}/\text{m}^3$.



Les exceptions en python

- Une exception est un signal envoyé par le programme lorsqu'une erreur ou une situation anormale survient, interrompant le flux normal d'exécution.
- Si le programme rencontre un problème qu'il ne peut pas résoudre immédiatement (division par zéro, fichier introuvable, ...), il lance une exception pour indiquer ce problème. Si personne ne la capture, le programme s'arrête avec un message d'erreur (traceback).
- En python, une exception est lancée avec le mot clé `raise` et gérée avec `try/except(/finally)`.
- Exemple: ouverture d'un fichier inexistant :

```
f = open("data.txt", "r", encoding="utf-8")

print("Le programme continue son exécution.")
```

```
C:\Users\maxim\Downloads>python3 exception_example.py
Traceback (most recent call last):
  File "C:\Users\maxim\Downloads\exception_example.py", line 1,
in <module>
    f = open("data.txt", "r", encoding="utf-8")
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory:
'data.txt'
```

```
try:
    f = open("data.txt", "r", encoding="utf-8")
except FileNotFoundError:
    print("Fichier introuvable.")

print("Le programme continue son exécution.")
```

```
C:\Users\maxim\Downloads>python3 exception_example.py
Fichier introuvable.
Le programme continue son exécution.
```

Le programme s'exécute toujours 😊

Sortie non prévue du programme! 😞

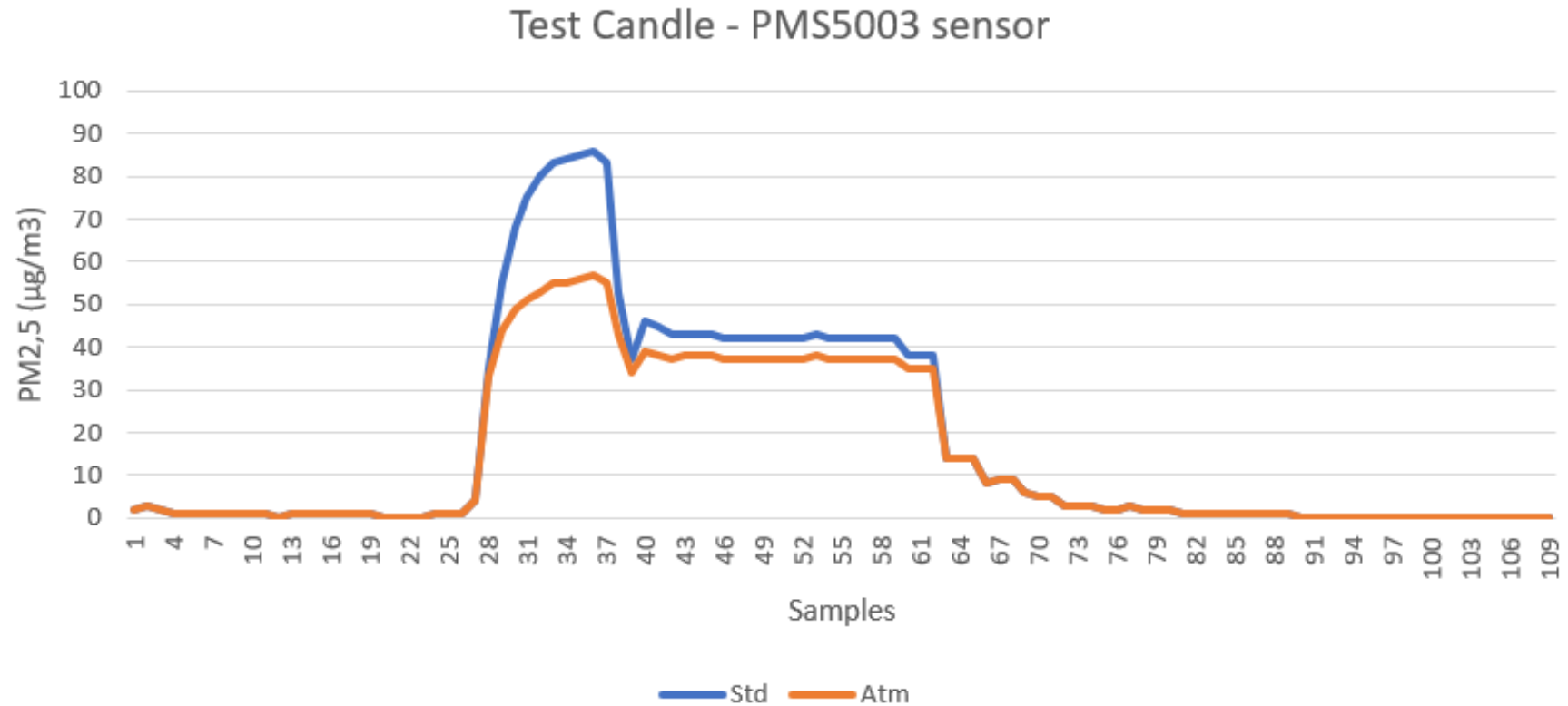
PMS5003 - Python

```
from pms5003 import PMS5003, ReadTimeoutError
import time

pms5003 = PMS5003() #enable PM sensor
time.sleep(0.5)

while True:
    try :
        data = pms5003.read() #read data from PM sensor
        print(data) #print a summary or only :
        print(f"{data.pm_ug_per_m3(1.0,False)} µg/m3") #print PM1.0 standard particle
        print(f"{data.pm_ug_per_m3(1.0,True)} µg/m3") #print PM1.0 atm environment
        print(f"{data.pm_ug_per_m3(2.5,False)} µg/m3") #print PM2.5 standard environment
        print(f"{data.pm_ug_per_m3(2.5,True)} µg/m3") #print PM2.5 atm particle
        print(f"{data.pm_ug_per_m3(10.0,False)} µg/m3") #print PM10 standard environment
        print(f"{data.pm_ug_per_m3(True)} µg/m3") #print PM10 atm particle
    except ReadTimeoutError:
        pms5003 = PMS5003()
```

PMS5003 - test

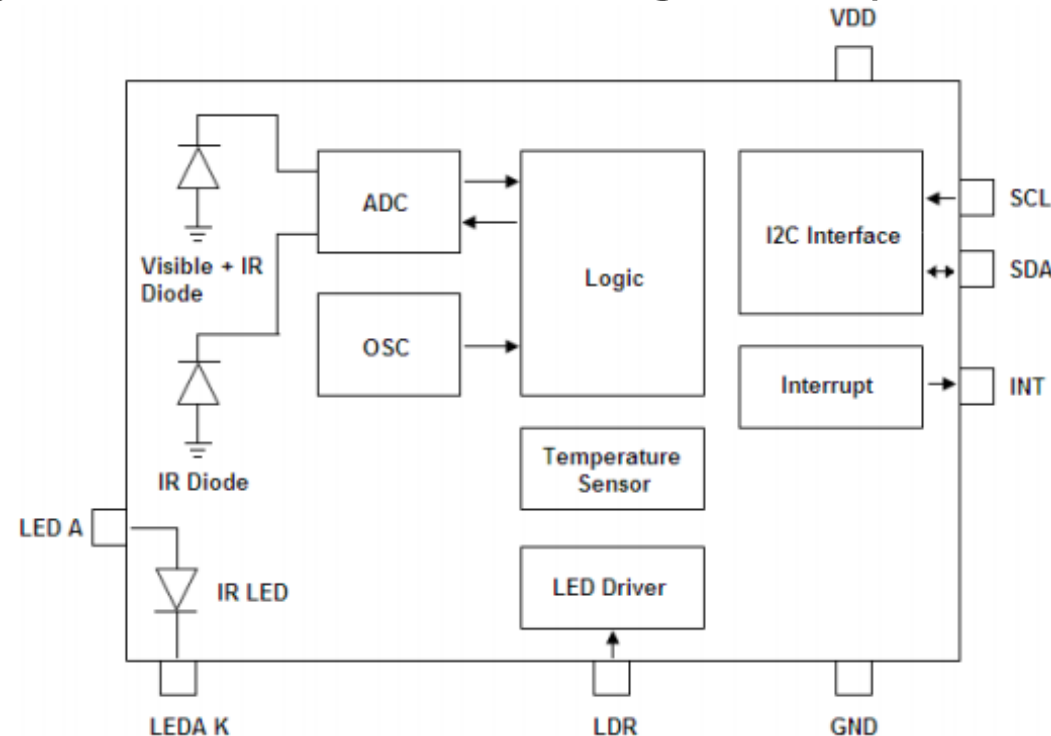


CAPTEURS

LTR-559ALS-01

LTR-559ALS-01

Le LTR-559ALS-01 est un capteur de lumière numérique I²C basse tension et un capteur de proximité dans un seul boîtier miniature. Ce capteur convertit l'intensité lumineuse en un signal de sortie numérique compatible avec une interface I²C. Il offre une réponse linéaire sur une large plage dynamique allant de 0,01 lux à 64 000 lux et convient parfaitement aux applications dans des environnements très lumineux. Grâce à son capteur de proximité intégré le LTR-559ALS-01 peut détecter des objets à une distance configurable par l'utilisateur.



LTR - Python

```
import time
from ltr559 import LTR559

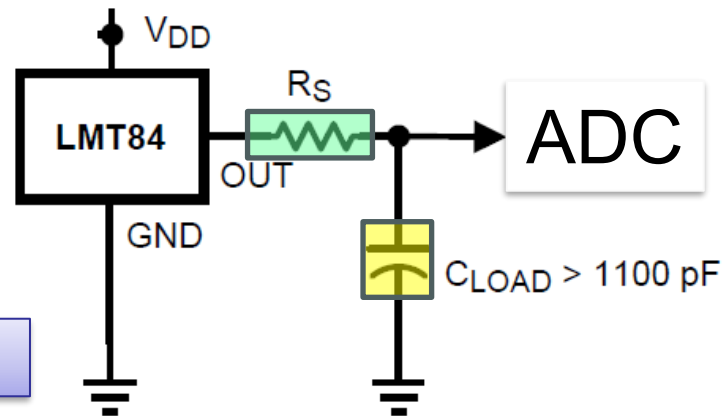
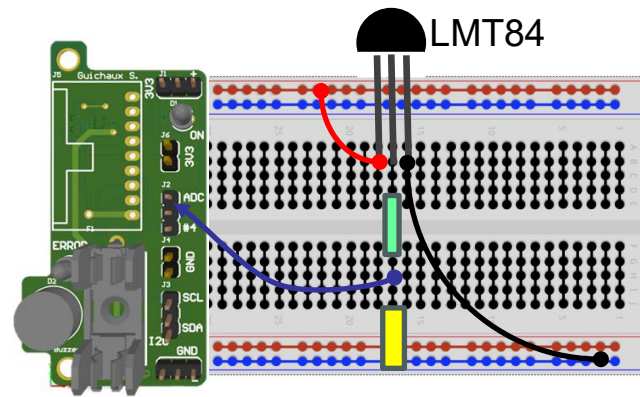
ltr559 = LTR559()

while True:
    ltr559.update_sensor()
    lux = ltr559.get_lux()
    print(f"Illuminance : {lux:.2f} lx")
    time.sleep(1.0)
```


LMT84

Le LMT84 est un capteur de température analogique. Sa plage de mesure s'étend de -50°C à 150°C avec une précision de $\pm 0,4^{\circ}\text{C}$. La sortie du capteur est une tension analogique proportionnelle à la température. La fonction de transfert du capteur est disponible dans son datasheet:

$$T = \frac{5,506 - \sqrt{(-5,506)^2 - 4 \times (-0,00176) \times (870,6 - V_{Temp[mV]}}}{2 \times (-0,00176)} + 30 [^{\circ}\text{C}]$$

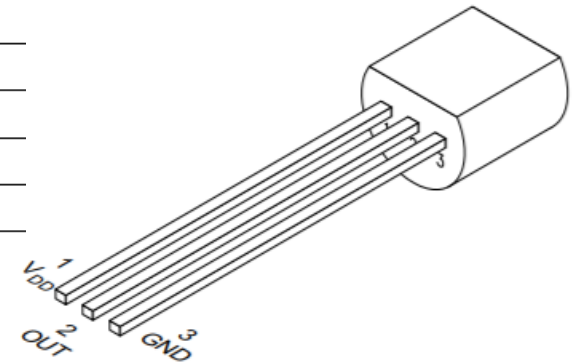


$$R_S = 1.2k\Omega$$
$$C_{LOAD} = 150nF$$

$T^{\circ} \leftrightarrow mV$

20	925
21	920
22	914
23	909
24	903
25	898
26	892
27	887
28	882
29	876

LP Package
3-Pin TO-92
(Top View)



Scale: 4:1

Rappel sur l'utilisation de l'ADC:

```
from enviroplus import gas
gas.enable_adc()
gas.set_adc_gain(4.096)

tension = gas.read_adc()
print(tension)
```

Datasheet : <https://www.ti.com/lit/ds/symlink/lmt84.pdf?ts=1643772541904>

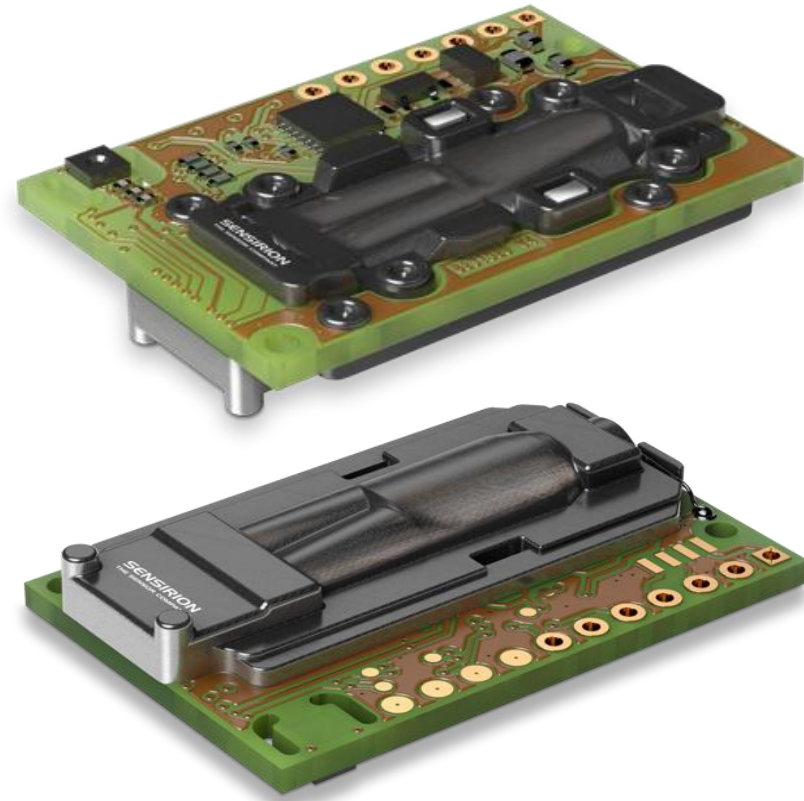
Remarque : $V_{DD} = 3.3V$

CAPTEURS

SCD30

SCD30

Il s'agit d'un capteur de CO₂ qui utilise une technologie optique. En plus de la mesure de CO₂, ce breakout embarque également un capteur d'humidité et de température. La précision de ce capteur est très bonne ce qui fait de lui un candidat idéal pour la conception d'un module de mesure professionnelle.

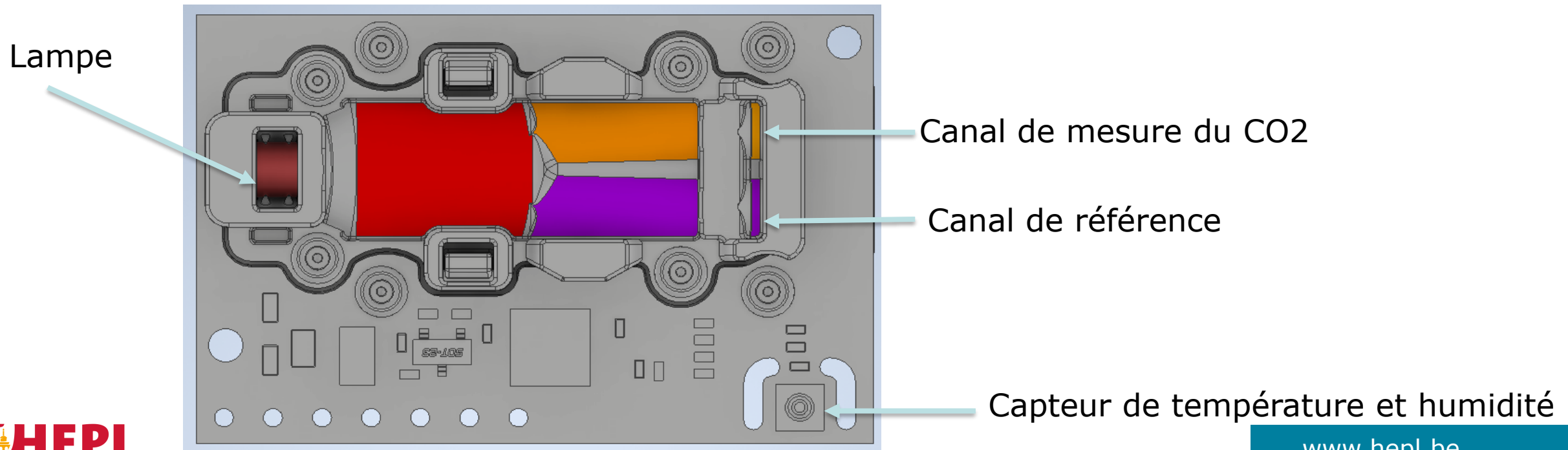


CO₂ Sensor Specifications

Parameter	Conditions	Value
CO ₂ measurement range	I2C, UART PWM	0 – 40'000 ppm 0 – 5'000 ppm
Accuracy ²	400 ppm – 10'000 ppm	± (30 ppm + 3%MV)
Repeatability ³	400 ppm – 10'000 ppm	± 10 ppm
Temperature stability ⁴	T = 0 ... 50°C	± 2.5 ppm / °C
Response time ⁵	$\tau_{63\%}$	20 s
Accuracy drift over lifetime ⁶	400 ppm – 10'000 ppm ASC field-calibration algorithm activated and SCD30 in environment allowing for ASC, or FRC field-calibration algorithm applied.	± 50 ppm

SCD30 - fonctionnement

Le SCD30 utilise la technologie NDIR à double canaux. Le CO_2 est une molécule qui absorbe très bien une certaine longueur d'onde dans l'infrarouge. La technique consiste donc à éclairer l'air ambiant avec une lampe infrarouge et mesurer l'absorption de cette longueur d'onde avec un capteur de lumière. Plus l'absorption est importante plus la concentration en CO_2 est élevée. Le SCD30 est équipé de deux canaux de détection, un pour la mesure d'air ambiant et l'autre pour la mesure d'un gaz de référence. Le 1^{er} canal détermine la concentration en CO_2 ambiante et le second permet de calibrer automatiquement le capteur.

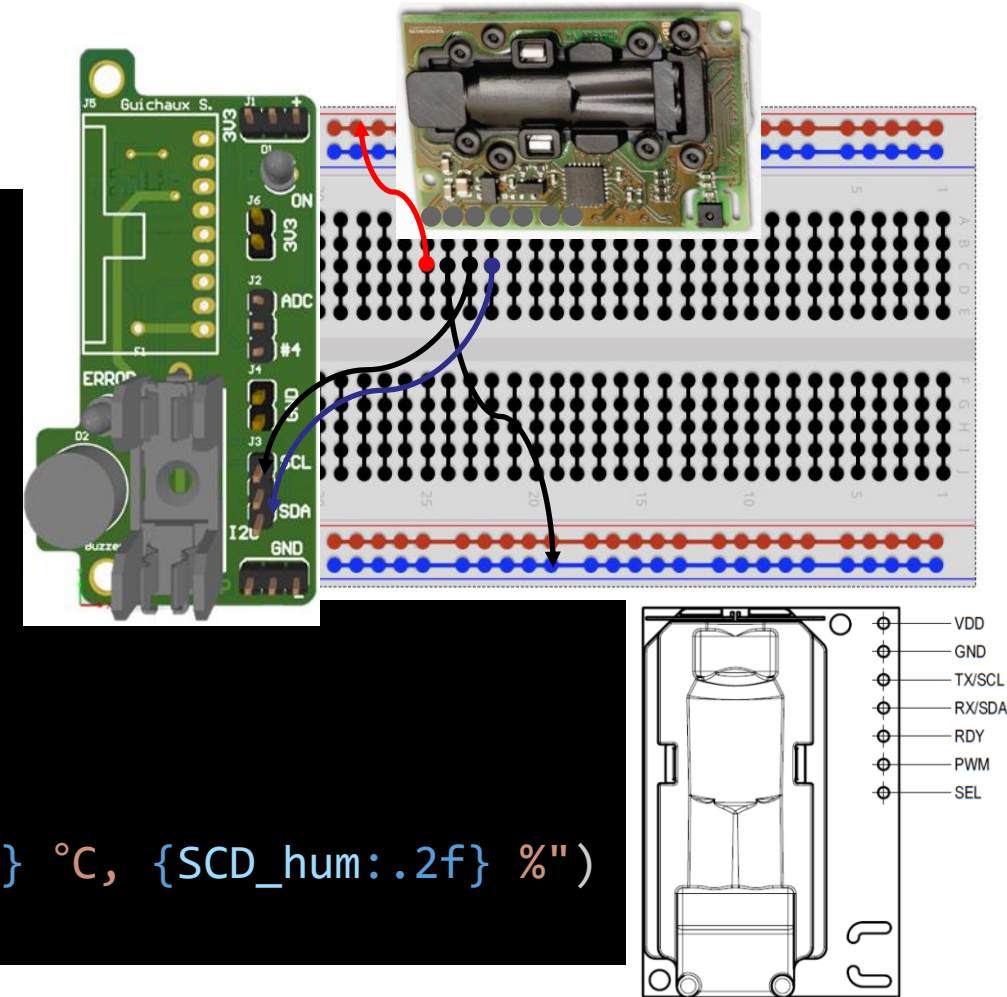


SCD30 - Python

```
import time
import adafruit_scd30
import board

scd=adafruit_scd30.SCD30(board.I2C())

while True:
    if scd.data_available:
        SCD_CO2= scd.CO2
        SCD_temp=scd.temperature
        SCD_hum= scd.relative_humidity
        print(f"{SCD_CO2:.2f} PPM, {SCD_temp:.2f} °C, {SCD_hum:.2f} %")
    time.sleep(1.0)
```



TRAITEMENT DE DONNÉES

TRAITEMENT DE DONNÉES

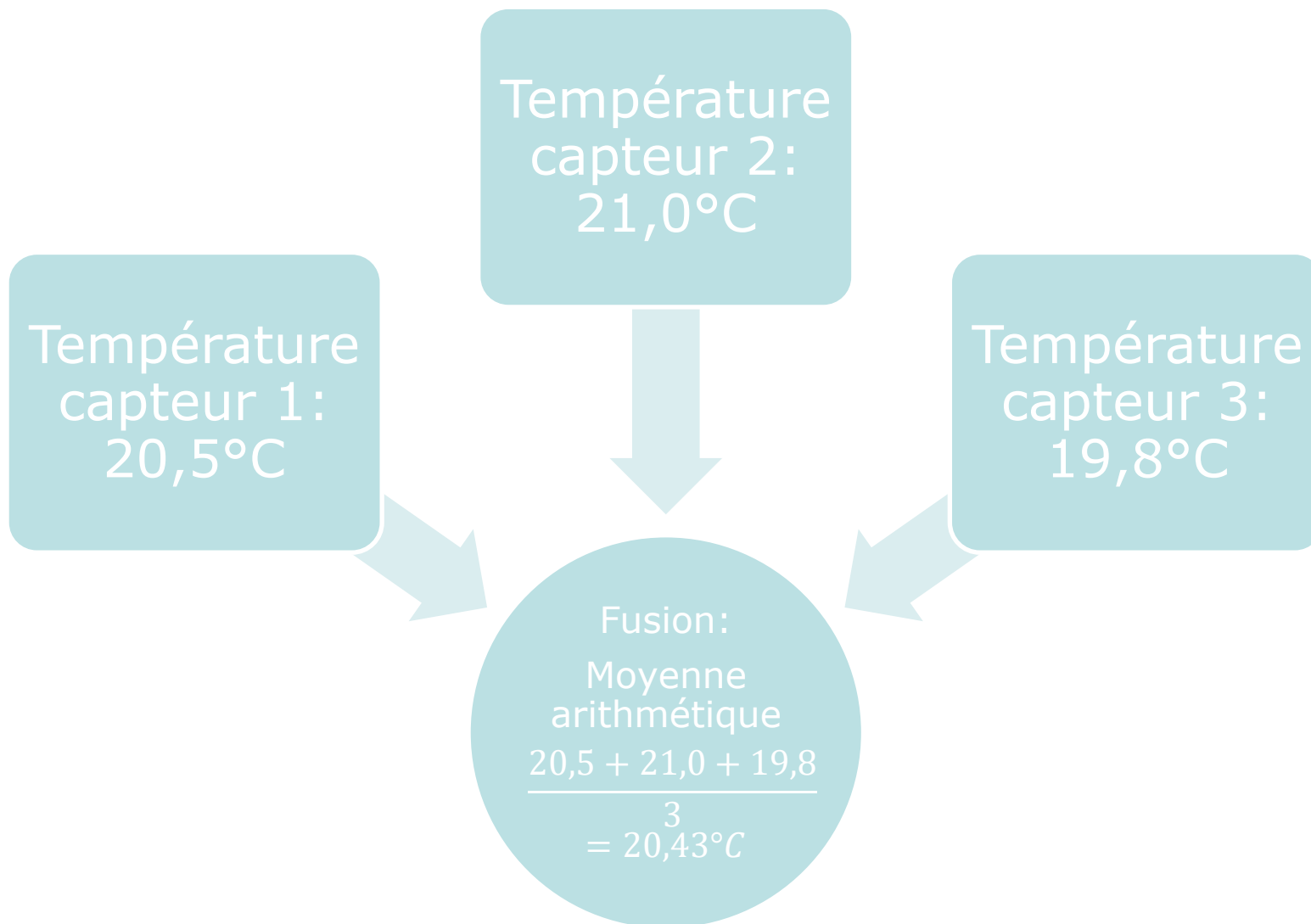
Fusion de données

Fusion de données

- La fusion de données désigne le processus consistant à **combiner des informations** provenant de plusieurs sources afin d'obtenir une représentation plus complète, cohérente et fiable d'un phénomène ou d'un système.
- L'idée est de réduire l'incertitude et d'améliorer la qualité des informations en exploitant la complémentarité des données issues de différents capteurs, bases ou systèmes.
- On distingue généralement 3 niveaux de fusion :
 - **Niveaux de fusion : Fusion de données brutes** (niveau capteur) : combine directement les mesures.
 - **Fusion de caractéristiques** : intègre des attributs extraits des données.
 - **Fusion décisionnelle** : combine des décisions ou interprétations issues de plusieurs systèmes.

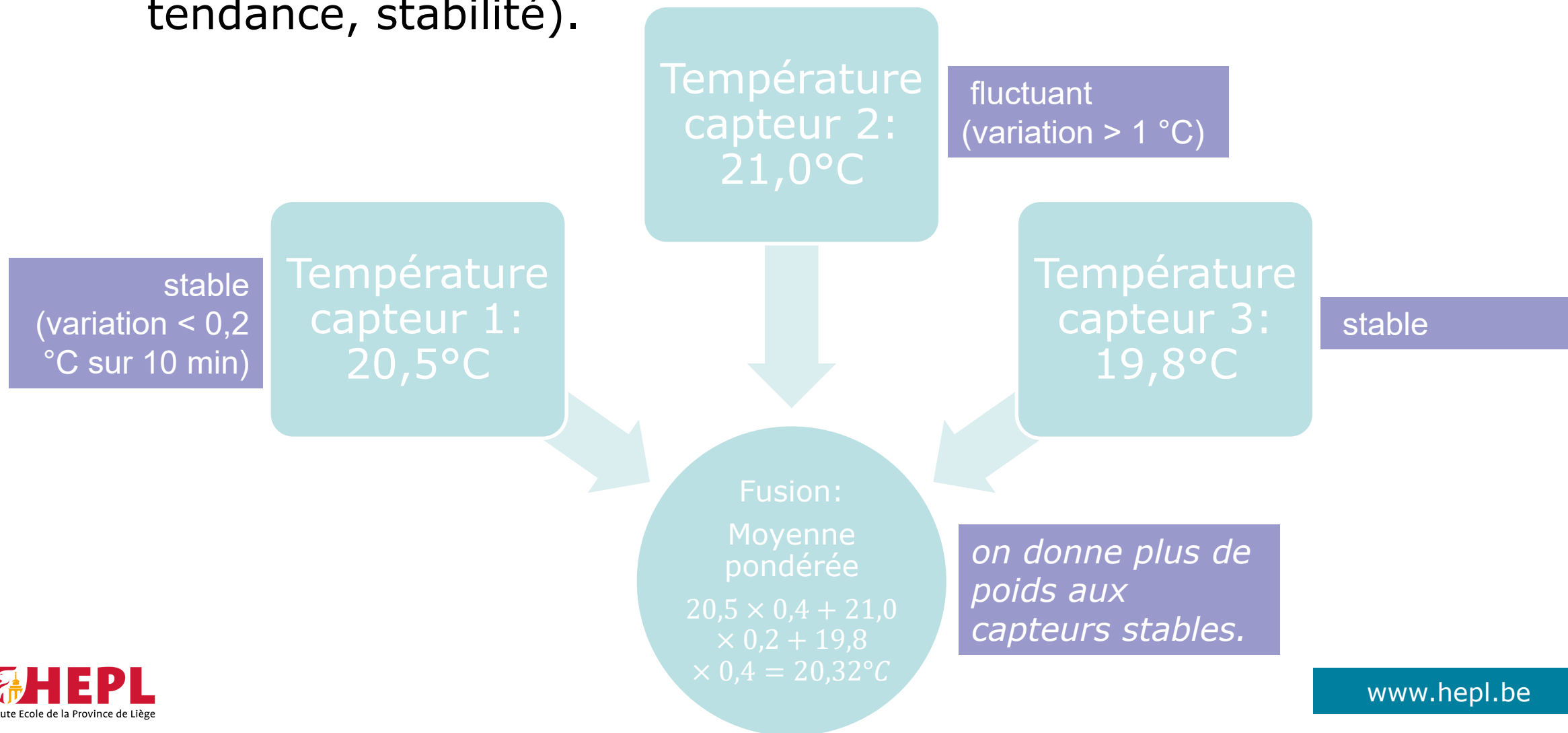
Fusion de données brutes

On combine directement les mesures des capteurs.



Fusion de caractéristiques

On extrait des informations supplémentaires avant la fusion (ex. tendance, stabilité).



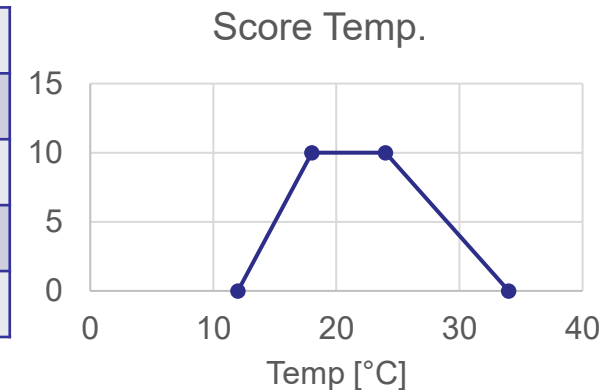
Fusion décisionnelle

On prend une décision à partir des résultats fusionnés ou des interprétations.

Température capteur : 21,0°C

Score de température dans un bureau:
L'OMS indique les niveaux suivant :
• Température : [18°C;24°C],

Temp. [°C]	Score
12	0
18	10
24	10
34	0



Fusion:
Score Temp.
 $t = 21^\circ\text{C} \rightarrow s = 10 \rightarrow \text{Bon}$

$$s = \begin{cases} \frac{5}{3}(t - 20) & \text{si } 12 < t < 18 \\ 10 & \text{si } 18 \leq t \leq 24 \\ 34 - t & \text{si } 24 < t < 34 \end{cases}$$

$$s - s_A = m \cdot (t - t_A), \quad \text{avec : } m = \frac{s_B - s_A}{t_B - t_A}$$

- Segment 1 : (12; 0), (18; 10) $\rightarrow s = 0 + \frac{10-0}{18-12}(t - 12) = \frac{10}{6}(t - 12)$
- Segment 2 : (18; 10), (24; 10) $\rightarrow s = 10 + \frac{10-10}{24-18}(t - 18) = 10$
- Segment 3 : (24; 10), (34; 0) $\rightarrow s = 10 + \frac{0-10}{34-24}(t - 24) = 34 - t$

[0;3[	[3;5[	[5;7[	[7;9[	[9;10]
Très mauvais	Mauvais	Moyen	Bon	Excellent

Fusion décisionnelle - python

- L'équation par morceau est codée en python avec des alternatives:

$$s = \begin{cases} \frac{5}{3}(t - 20) & \text{si } 12 < t < 18 \\ 10 & \text{si } 18 \leq t \leq 24 \\ 34 - t & \text{si } 24 < t < 34 \end{cases}$$

- De même que l'échelle :

[0;3[	[3;5[	[5;7[	[7;9[	[9;10]
Très mauvais	Mauvais	Moyen	Bon	Excellent

- Remarquez la prise en compte des valeurs extérieures aux bornes prévues par écrêtage ou par exception.

```
def score_temp(t):  
    if t < 12:  
        return 0  
    elif 12 <= t < 18:  
        return (5/3)*(t-20)  
    elif 18 <= t <= 24:  
        return 10  
    elif 24 < t <= 34:  
        return 34-t  
    else :  
        return 0
```

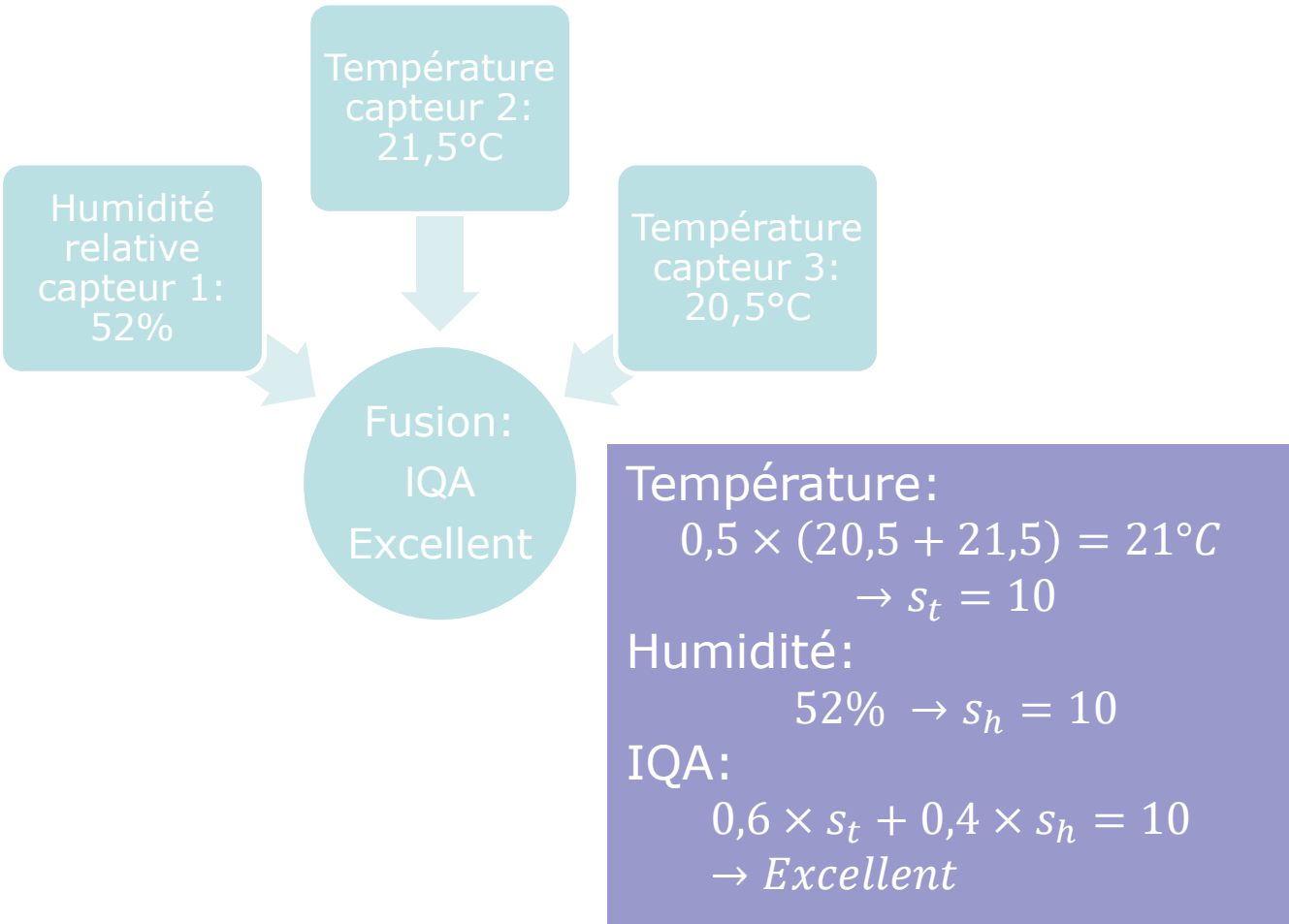
```
def score_label(s):  
    if s<0:  
        raise ValueError("Le score doit être entre 0 et 10.")  
    if 0<=s<3:  
        return "Très mauvais"  
    elif 3<=s<5:  
        return "Mauvais"  
    elif 5<=s<7:  
        return "Moyen"  
    elif 7<=s<9:  
        return "Bon"  
    elif 9<=s<=10:  
        return "Très bon"  
    else :  
        raise ValueError("Le score doit être entre 0 et 10.")
```

```
t = 21  
s = score_temp(t)  
print(f"Le score pour une température de {t}°C est {s}.")  
print(f"Ce score de température est {score_label(s)}.")
```

```
C:\Users\maxim\Downloads>python3 score_temp.py  
Le score pour une température de 21°C est 10.  
Ce score de température est Très bon.
```

Fusion multi-niveaux

On mélange plusieurs niveaux de fusion.

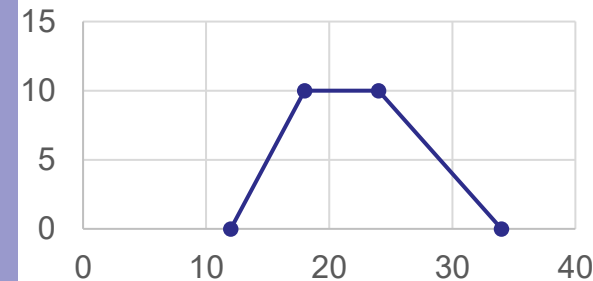


Indice de qualité de l'air dans un bureau:
 L'OMS indique les niveaux suivant :

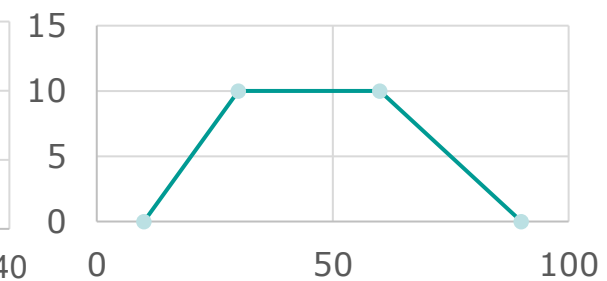
- Température : $[18^\circ\text{C}; 24^\circ\text{C}]$,
- Humidité relative : $[30\%; 60\%]$.

Temp. [$^\circ\text{C}$]	Hr [%]	Score
12	10	0
18	30	10
24	60	10
34	90	0

Score Temp.



Score Hum.



[0;3[	[3;5[	[5;7[	[7;9[	[9;10]
Très mauvais	Mauvais	Moyen	Bon	Excellent

TRAITEMENT DE DONNÉES

Listes

Gestion de données - python

- L'acquisition et la manipulation d'un volume de données important dans un code nécessite des structures de données adaptées.
- L'une des plus pratique et des plus répandue est la liste.
- Une liste est une structure de données qui permet de stocker plusieurs valeurs dans une seule variable.
 - Les éléments sont ordonnés (ils ont une position).
 - Les éléments sont modifiables.
 - Une liste peut contenir des éléments de types différents (entiers, flottants, chaînes, autres listes, ...) bien que cela ne soit pas encouragé.

■ Exemple : `liste_temp = [18.5, 19.0, 20.0, 21.5]`

Les listes en python

- Accès aux éléments d'une liste:
Chaque élément possède un indice (index) qui commence à 0.

```
liste_temp = [18.5, 19.0, 20.0, 21.5]
print(liste_temp[0])    # 18.5
print(liste_temp[2])    # 20.0
print(liste_temp[-1])   # 21.5 (dernier élément)
```

- Modification d'un élément d'une liste:

```
liste_temp[2] = 20.5 # -> [18.5 19.0 20.5 21.5]
```

- Ajout d'un élément à la fin de la liste :

```
liste_temp.append(22.0) # -> [18.5 19.0 20.5 21.5 22.0]
```

- Insertion d'un élément à une position précise:

```
liste_temp.insert(1, 19.5) # -> [18.5 19.5 19.0 20.5 21.5 22.0]
```

- Suppression d'un élément à une position précise :

```
liste_temp.pop(3) # -> [18.5 19.5 19.0 21.5 22.0]
```


Les listes en python

- Obtention de la taille d'une liste:

```
nombre = len(liste_temp) # -> nombre = 5
```

- Parcours d'une liste par élément:

```
for temp in liste_temp:  
    print(temp)
```

- Parcours d'une liste par index:

```
for i in range(len(liste_temp)):  
    print(f"Index {i}: {liste_temp[i]}")
```

- Exemple : calcul d'une moyenne sur les éléments d'une liste :

```
def Average(l):  
    sum=0  
    for i in range(0,len(l)):  
        sum+=l[i]  
    return sum/len(l)  
  
liste_temp = [18.5, 19.0, 20.0, 21.5]  
avg_temp = Average(liste_temp)  
print(f"Average temperature: {avg_temp} °C")
```

Average temperature: 19.75 °C

TRAITEMENT DE DONNÉES

Dictionnaires

Un index non numérique ?

- L'usage des listes est très pratique mais il arrive parfois que leur accès par un index numérique ne soit pas le plus adapté par rapport à nos besoins.
- On a parfois besoin d'associer une information non numérique à une autre et dans ce cas on a plutôt recours à un dictionnaire.
- Un dictionnaire est une structure de données qui permet de stocker des informations sous la forme de paires clé → valeur.
 - Chaque clé est unique,
 - Les valeurs peuvent être de n'importe quel type,
 - Les éléments sont modifiables,
 - L'ordre d'insertion est conservé (python 3.7+).
- Exemple :

```
data_dict = {"temperature": 21.0, "humidity": 45.0, "pressure": 1013.25}
```


temperature, humidity et pressure sont des clés de type « string »
21.0;45.0 et 1013.25 sont des valeurs.

Les dictionnaires en python

- Accès aux valeurs d'un dictionnaire:
 - ⚠ Si la clé n'existe pas, une exception est levée.

```
data_dict = {"temperature": 21.0, "humidity": 45.0, "pressure": 1013.25}
print(data_dict["temperature"])
print(data_dict["humidity"])
```

- Modification ou ajout d'un élément d'un dictionnaire:

```
data_dict["humidity"] = 50.0 # si la clé existe, remplacement de la valeur
data_dict["co2"] = 600.0     # si la clé n'existe pas, creation d'une nouvelle paire
```

- Vérification de l'existence d'une clé:

```
if "co2" in data_dict:
    print("CO2 level:", data_dict["co2"])
```

- Parcours d'un dictionnaire :

```
for key, value in data_dict.items(): # par paire
    print(f"{key}: {value}")
for key in data_dict.keys():         # par clé
    print("Key:", key)
for value in data_dict.values():     # par valeur
    print("Value:", value)
```

TRAITEMENT DE DONNÉES

Moyenne glissante

Moyenne glissante

- Il est fréquent que les données fournies par les capteurs soient polluées par du bruit non désiré.
- Il est alors utile de nettoyer ou lisser les données collectées avec un filtre "passe-bas". Il en existe plusieurs types mais l'un des plus simple à mettre en oeuvre est la **moyenne glissante**.
- Principe:
On choisit une taille de fenêtre (ex. 3, 5 ou 10 points). Pour chaque point, on calcule la moyenne des valeurs dans cette fenêtre. La fenêtre se déplace d'un pas à chaque calcul.
- Formule :
$$\text{Moyenne glissante} = \frac{x_i + x_{i+1} + \dots + x_{i+n-1}}{n}$$

où n est la taille de la fenêtre et x_i = valeur à l'index i .

Moyenne glissante – python

- On utilise une liste raw pour stocker les valeurs acquises par un capteur (ici, simulé par des valeur aléatoires avec la fonction SensorValue).
- La liste raw, initialisée vide, se remplit avec une nouvelle mesure de température à chaque tour de boucle :

```
def SensorValue(min, max, steps):  
    return random.randint(steps*min, steps*max)/steps  
  
def StoreSample(l, value):  
    l.append(value)  
  
raw=[]  
  
while True:  
    latest = SensorValue(min,max,steps)  
    StoreSample(raw, latest)  
    time.sleep(1)
```

Moyenne glissante – python

- On utilise une autre liste `moving` pour stocker les `N` dernières valeurs acquises, sur laquelle sera calculée la moyenne glissante.
- La liste `moving`, initialisée vide, se remplit jusqu'à atteindre la longueur `N`, ensuite l'élément le plus ancien est supprimé (`pop(0)`) pour maintenir une longueur `N` constante.
- Le calcul de la moyenne est fait sur `N` éléments à chaque tour de boucle. La variable `mavg` contient le résultat de la moyenne glissante.

```
def SensorValue(min, max, steps):  
    return random.randint(steps*min, steps*max)/steps  
  
def StoreSample(l, value):  
    l.append(value)  
  
moving=[]  
raw=[]  
n=5  
while True:  
    latest = SensorValue(min,max,steps)  
    StoreSample(raw, latest)  
    mavg = MovingAverage(moving, latest, n)  
    time.sleep(1)
```

```
def Average(l):  
    sum=0  
    for i in range(0,len(l)):  
        sum+=l[i]  
    return sum/len(l)  
  
def MovingAverage(l, latest, size):  
    if len(l) == size:  
        l.pop(0)  
    l.append(latest)  
    return Average(l)
```


Moyenne glissante – python

- On utilise une dernière liste smooth pour stocker les valeurs calculées par la fonction MovingAverage.
- La liste smooth contient donc les valeurs de la moyenne glissante.

```
def SensorValue(min, max, steps):  
    return random.randint(steps*min, steps*max)/steps  
  
def StoreSample(l, value):  
    l.append(value)  
  
moving=[]  
raw=[]  
smooth=[]  
n=5  
while True:  
    latest = SensorValue(min,max,steps)  
    StoreSample(raw, latest)  
    mavg = MovingAverage(moving, latest, n)  
    StoreSample(smooth, mavg)  
    time.sleep(1)
```

```
def Average(l):  
    sum=0  
    for i in range(0,len(l)):  
        sum+=l[i]  
    return sum/len(l)  
  
def MovingAverage(l, latest, size):  
    if len(l) == size:  
        l.pop(0)  
    l.append(latest)  
    return Average(l)
```

Moyenne glissante – python

- Avec un graphique matplotlib, on obtient ce code :

```
import random
import time
import datetime
import matplotlib.pyplot as plt

def Average(l):
    sum=0
    for i in range(0,len(l)):
        sum+=l[i]
    return sum/len(l)

def MovingAverage(l, latest, size):
    if len(l) == size:
        l.pop(0)
    l.append(latest)
    return Average(l)

def SensorValue(min, max, steps):
    return random.randint(steps*min,
steps*max)/steps

def StoreSample(l, value):
    l.append(value)

def UpdatePlot(xaxis, raw, smooth,
legend, initDone):
    plt.plot(xaxis, raw, '--b')
    plt.plot(xaxis, smooth, '-r')
    plt.legend(legend)

    if not initDone:
        plt.show(block=False)
        plt.grid()
        initDone = True
    else:
        plt.draw()

    plt.pause(1)

    return initDone

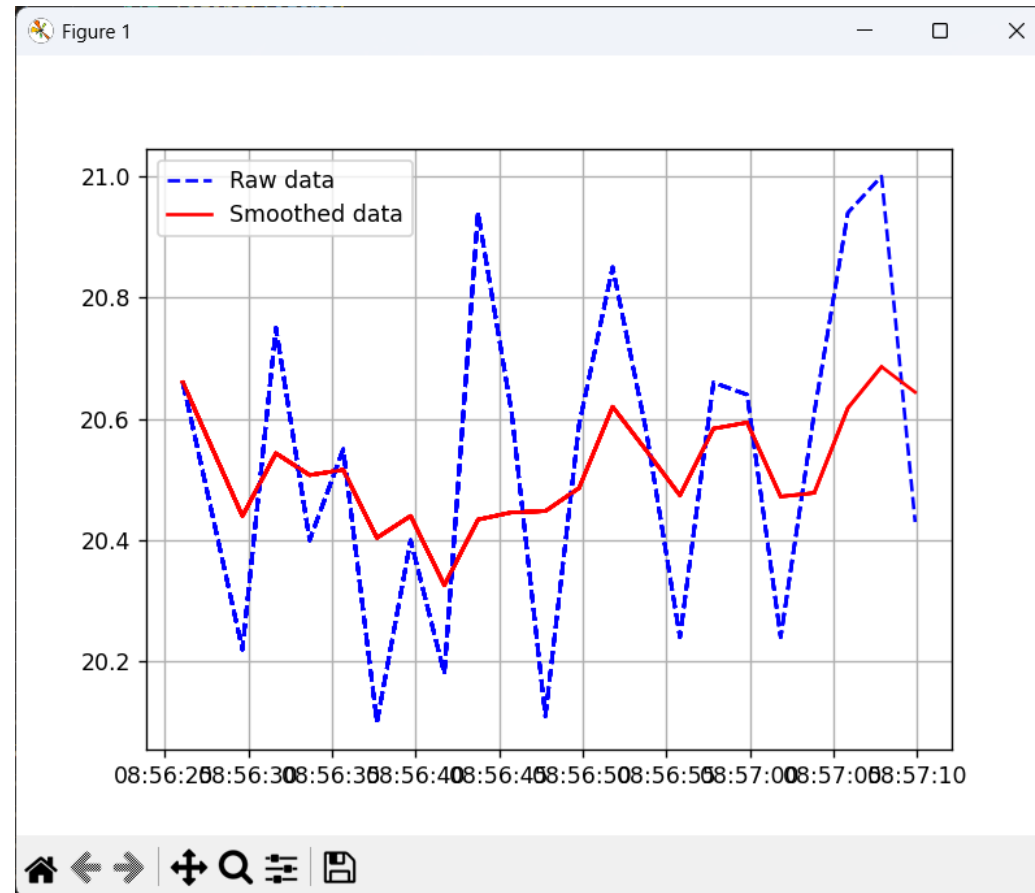
def main(n, min, max, steps):
    moving=[]
    raw=[]
    smooth=[]
    xaxis=[]
    legend=['Raw data', 'Smoothed
data']
    initDone=False

    while True:
        latest =
SensorValue(min,max,steps)
        StoreSample(raw, latest)
        StoreSample(smooth,
MovingAverage(moving, latest, n))
        StoreSample(xaxis,
datetime.datetime.now())
        initDone = UpdatePlot(xaxis,
raw, smooth, legend, initDone)
        time.sleep(1)

    if __name__ == '__main__':
        main(5, 20, 21, 100)
```

Moyenne glissante – python

- Ce qui donne pour N=5:



- On observe bien que la moyenne glissante atténue les variations rapides de température.

Moyenne glissante – python

- Quel est l'effet de la taille de la fenêtre glissante ?
Pour ce faire modifions notre code pour calculer simultanément la moyenne glissante avec N=3, 5 et 9.
- En utilisant des dictionnaires, les modifications de code sont minimales :

```
def UpdatePlot(xaxis, raw, smooth, legend, initDone):
    plt.plot(xaxis, raw, '--b')
    plt.plot(xaxis, smooth["3"], '-r')
    plt.plot(xaxis, smooth["5"], '-g')
    plt.plot(xaxis, smooth["9"], '-m')
    plt.legend(legend)

    if not initDone:
        plt.show(block=False)
        plt.grid()
        initDone = True
    else:
        plt.draw()

    plt.pause(1)

    return initDone

def main(min, max, steps):
    moving={"3":[], "5":[], "9":[]}
    raw=[]

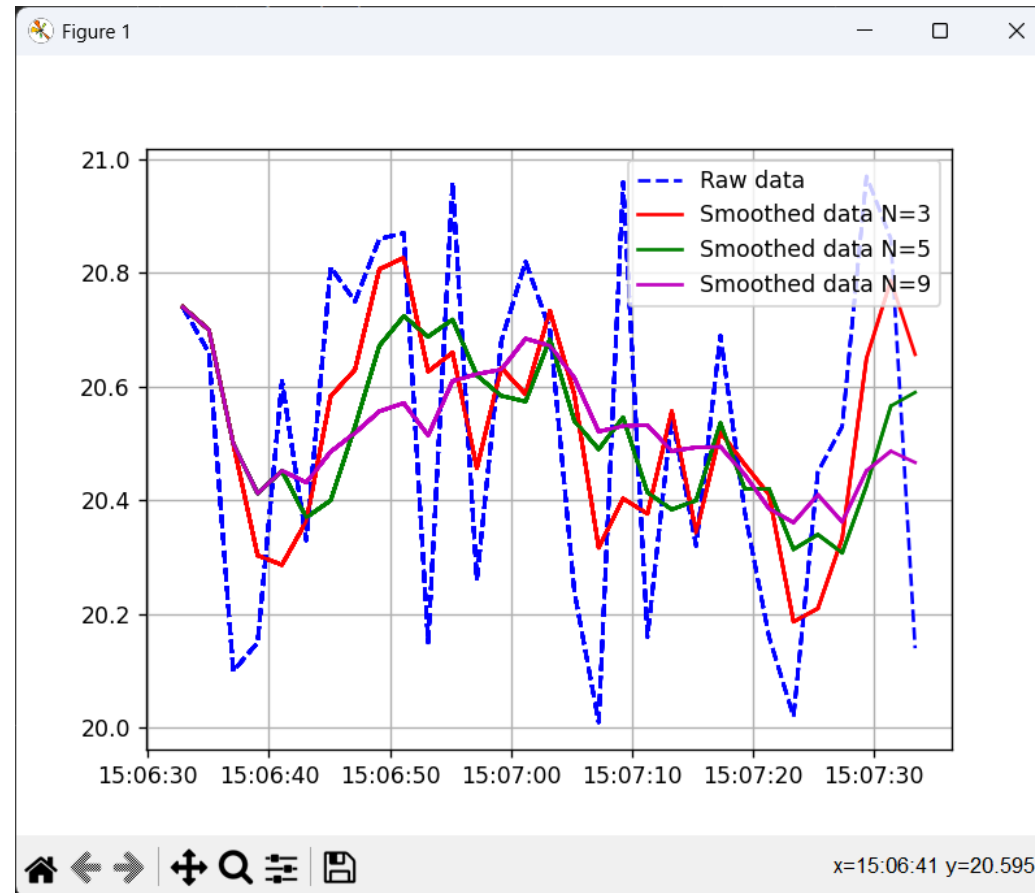
    smooth={"3":[], "5":[], "9":[]}
    xaxis=[]
    legend=['Raw data', 'Smoothed data N=3', 'Smoothed data N=5',
'Smoothed data N=9']
    initDone=False

    while True:
        latest = SensorValue(min,max,steps)
        StoreSample(raw, latest)
        StoreSample(smooth["3"], MovingAverage(moving["3"], latest,
3))
        StoreSample(smooth["5"], MovingAverage(moving["5"], latest,
5))
        StoreSample(smooth["9"], MovingAverage(moving["9"], latest,
9))
        StoreSample(xaxis, datetime.datetime.now())
        initDone = UpdatePlot(xaxis, raw, smooth, legend, initDone)
        time.sleep(1)

    if __name__ == '__main__':
        main(20, 21, 100)
```

Moyenne glissante – python

- Ce qui donne pour N=3, 5 et 9:



- On observe que la taille de la fenêtre glissante atténue d'autant plus les variations rapides que sa taille est grande (→ moyenne totale).

Moyenne glissante – python

- Peut-on rendre notre code indépendant de valeurs de N ? Oui, bien sûr ;-)

```
def UpdatePlot(xaxis, raw, smooth, legend, initDone):
    plt.plot(xaxis, raw, '--b')
    for n, mavg_curve in smooth.items():
        plt.plot(xaxis, mavg_curve, '-')
    plt.legend(legend)

    if not initDone:
        plt.show(block=False)
        plt.grid()
        initDone = True
    else:
        plt.draw()

    plt.pause(1)

    return initDone

def main(nn, min, max, steps):
    moving = {}
    smooth = {}
    for n in nn:
        moving[n]=[]
        smooth[n]=[]
    raw=[]
    xaxis=[]

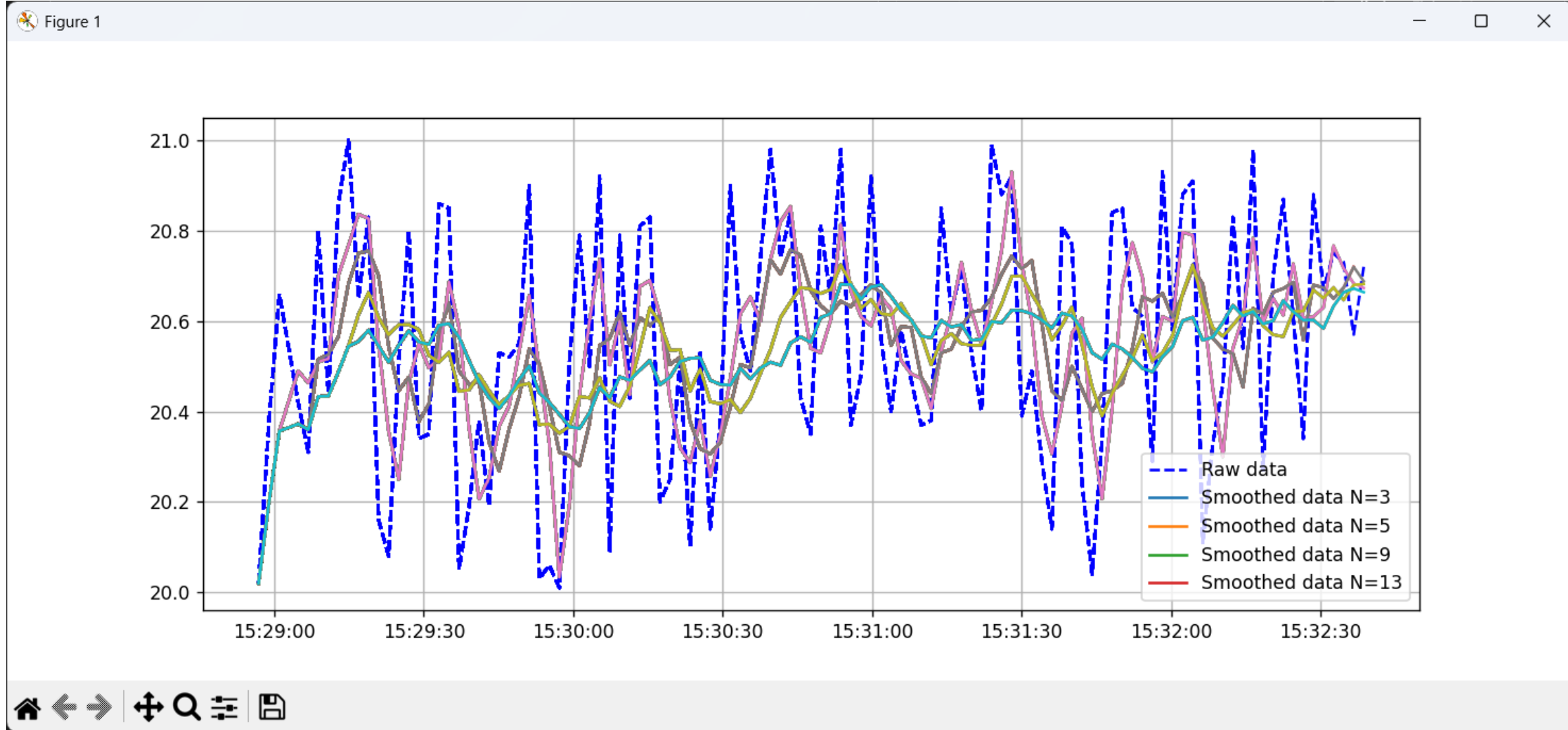
    legend=['Raw data']
    for n in nn:
        legend.append(f"Smoothed data N={n}")
    initDone=False

    while True:
        latest = SensorValue(min,max,steps)
        StoreSample(raw, latest)
        for n in nn:
            StoreSample(smooth[n], MovingAverage(moving[n], latest,
n))
        StoreSample(xaxis, datetime.datetime.now())
        initDone = UpdatePlot(xaxis, raw, smooth, legend, initDone)
        time.sleep(1)

if __name__ == '__main__':
    main([3,5,9,13], 20, 21, 100)
```

Il suffit alors d'appeler main avec la liste de N voulus.

Moyenne glissante – python



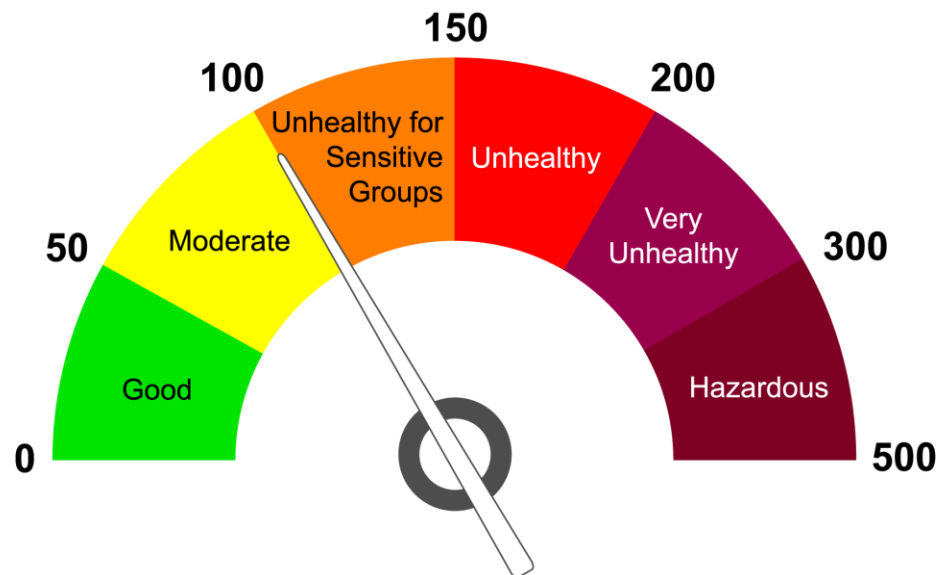
PROJET FINAL

Projet - Objectifs

- Interfacer les **6 capteurs** environnementaux du projet,
- Déterminer un **indice global de qualité de l'air** -> fusion de données (IQA),
- Afficher en temps réel les données récoltées sous forme de tableaux et graphiques dans **NodeRed**,
- Examiner les résultats à l'issue d'une **campagne de mesure** et en dégager les tendances en matière de qualité de l'air.

Fusion de données - IQA

- Établir un indice de la qualité de l'air à partir des mesures des capteurs du projet.
- Vous déterminez :
 - Les mesures pertinentes à prendre en compte (au minimum 2 différentes),
 - Les niveaux acceptables et « dangereux » de chaque mesure (recherche biblio),
 - La répartition des indicateurs de votre échelle (bien-moyen-...),
 - La dimension de votre échelle (0-100, 0-500, ...).
- Il n'y a pas de mauvais choix mais chacun devra être justifié dans votre rapport de projet.



Cahier des charges du projet

1. Interfaçage des capteurs (réaliser un schéma bloc dans votre rapport) :

- BME280 : température, pression et humidité relative,
- LTR559 : lumière ambiante,
- SPH0645LM4H-B : microphone,
- PMS5003 : Particules fines,
- SCD30 : CO₂,
- LMT84 : température,

2. Software :

- Enregistrement des données au format CSV, la période est à votre discrétion.
- Calcul d'un indice de la qualité de l'air via **fusion des données** des capteurs pertinents.
- Calcul d'une **moyenne glissante** pour les données d'au moins un capteur.
- Interface graphique NodeRed qui affiche en temps réel les mesures et calculs via messages MQTT.
- Interface graphique NodeRed qui contrôle la prise de mesure du programme python (au moins 2 paramètres).

3. Campagne de mesure :

- Mesure sur une période d'au moins 30 minutes.
- Prise de mesures des capteurs et calcul de l'indice de la qualité de l'air.
- Analyse des résultats via la réalisation de graphiques des mesures.
- Recherche sur les niveaux acceptables des différents composés (gaz, PM,...) et caractéristiques environnementales (température, pression,...) mesurés (indiquez vos sources).

- Consultez le document PDF complet disponible dans le canal du projet (Projet MSEQ2 Session1.pdf)